Group Coursework Submission Form

Specialist Masters Programme

Please list all names of group members: (Surname, first name) 1. Doshi Ishan 2. Gehlot Purvi 3. Dash Debasmita	4. Jain Tithi 5. 6. 7. **GROUP NUMBER:1**
MSc in: Business Analytics	
Module Code: SMM636	
Module Title: Machine Learning	
Word Count : 910	
Lecturer: Dr. Rui Zhu	**Submission Date: 03/04/2025**

Declaration:

By submitting this work, we declare that this work is entirely our own except those parts duly identified and referenced in my submission. It complies with any specified word limits and the requirements and regulations detailed in the coursework instructions and any other relevant programme and module documentation. In submitting this work we acknowledge that we have read and understood the regulations and code regarding academic misconduct, including that relating to plagiarism, as specified in the Programme Handbook. We also acknowledge that this work will be subject to a variety of checks for academic misconduct.

We acknowledge that work submitted late without a granted extension will be subject to penalties, as outlined in the Programme Handbook. Penalties will be applied for a maximum of five days lateness, after which a mark of zero will be awarded.

Marker's Comments (if not being marked on-line):

Deduction for Late Submission:

Final Mark: %

Contents

Introduction 3
Principal Component Analysis 3
Variance Explained 3
Component Loadings 3
Visualization in PCA Space 4
Clustering Analysis 5
Methodology Justification 5
Optimal Cluster Determination 5
Cluster Characteristics 6
Interpretation and Insights 7
Performance of ChatGPT-4o 8
Similarities and Difference 9
Reliability and usefulness 9
Appendices 10

Introduction

This analysis examines the structure and patterns within a dataset of 50 top-rated IMDb movies using dimensionality reduction techniques. By applying Principal Component Analysis (PCA) to the numerical attributes of these films (Runtime, Rating, Votes, and Revenue), we uncover latent relationships between commercial success, critical reception, and viewer engagement. These insights can inform strategic decision-making in film production, marketing, and distribution channels.

Principal Component Analysis

Variance Explained

The PCA results[Table – 1] demonstrate that the first two principal components collectively account for 72.87% of the total variance in the dataset, indicating strong linear correlations among the variables. This suggests that high performance in one attribute often coincides with success in others, revealing an underlying pattern characterizing successful movies.

Component	Variance Explained	Cumulative Variance
PC1	43.51%	43.51%
PC2	29.36%	72.87%
PC3	19.31%	92.18%
PC4	7.82%	100.00%

Table I: Variance Distribution across Principal Components

Component Loadings

The loading coefficients reveal how each original variable contributes to the principal components:

Variable	PC1	PC2	PC3	PC4
Runtime (Minutes)	0.3567	-0.5749	-0.6932	-0.2487
Rating	0.4505	-0.4388	0.7097	-0.3175
Votes	0.6844	0.1458	-0.0249	0.7140
Revenue (Millions)	0.4488	0.6751	-0.1236	-0.5724

Table II: Variable Contribution to Principal Components

PC1 exhibits strong positive coefficients across all variables, with the highest loading on Votes (0.68), followed by Rating (0.45) and Revenue (0.45). This component represents a "Market Impact" dimension, capturing films with broad audience reach, critical acclaim, and commercial success.

PC2 reveals a more nuanced interplay, with positive loading on Revenue (0.68) but negative loadings on Runtime (-0.57) and Rating (-0.44). This component distinguishes between commercially successful films favoring accessibility (shorter runtime, wider appeal) versus critically acclaimed films prioritizing artistic depth (longer runtime, higher ratings).

Visualization in PCA Space

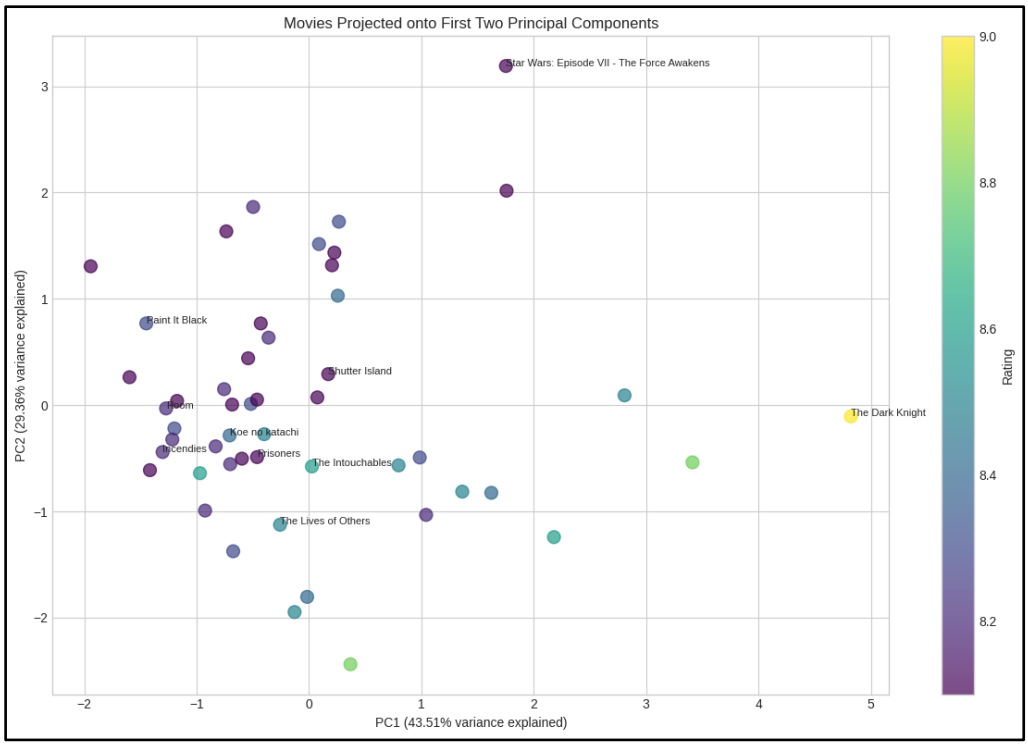


Figure I: Movies Projected on PC1 and PC2

The PCA visualization reveals four distinct film clusters:

Quadrant	Characteristics	Examples	Strategic Implications
Upper-Right (High PC1, High PC2)	High Votes and Revenue, moderate-to-high Rating, longer Runtime	Inception, The Dark Knight, Interstellar, The Avengers	Blockbusters with strong audience engagement and commercial returns
Lower-Right (High PC1, Low PC2)	High Rating and Votes, lower Revenue, longer Runtime	Whiplash, Dangal, The Intouchables, Kimi no na wa	Critically acclaimed films with strong audience approval but moderate revenue
Upper-Left (Low PC1, High PC2)	High Revenue, low Rating and Votes, shorter Runtime	Frozen, Minions, Twilight Saga: Breaking Dawn Part 2	Commercially successful films with limited critical approval, driven by franchise strength
Lower-Left (Low PC1, Low PC2)	Low across all variables: Rating, Votes, Revenue, shorter Runtime	Paint It Black, A Separation, The Secret in Their Eyes	Niche or independent films with limited exposure and modest performance

Table III: Visualization in PCA Space

This multi-dimensional representation challenges the simplistic notion of a single formula for successful movies. Instead, different films succeed through varied combinations of attributes that appeal to different market segments, providing valuable insights for strategic positioning in film production and marketing.

Clustering Analysis

After reducing the dataset's complexity using PCA, clustering was employed to explore and categorize the structural patterns within the simplified feature space. Clustering groups having similar movies based on numerical patterns in Runtime, Rating, Votes, and Revenue . Our objective is to identify hidden market segments (e.g., blockbusters vs niche films) using K-means and hierarchical clustering, building on PCA's dimensionality reduction (73% variance captured by PC1-PC2).

Methodology Justification

K-means was selected for its efficiency with numerical data and clear cluster interpretation via centroids showing distinct revenue/rating groups. Hierarchical clustering complemented this by revealing natural groupings through its dendrogram (output: 3 main branches at distance=7.5), aligning with k-means' elbow at k=3 (plot: sharp bend at k=3). This dual approach follows best practices for validation - k-means optimizes Euclidean distances (mathematical basis: n_init=10 trials), while hierarchical provides interpretable taxonomy via Ward's linkage.

Optimal Cluster Determination

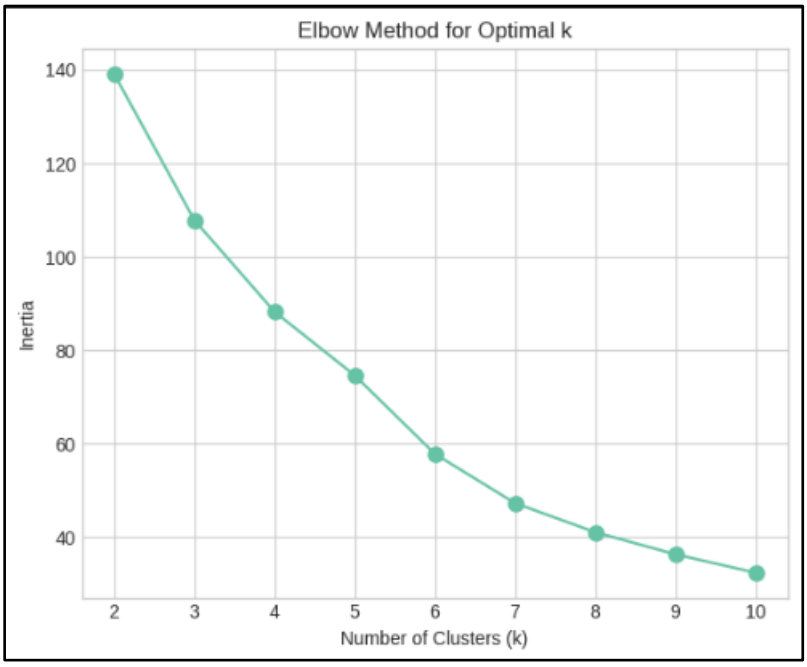


Figure II: Elbow Method for Optimal K

The Elbow Method analysis (Figure III) determined optimal clusters by tracking inertia—the sum of squared distances to cluster centroids. The sharp decline in inertia plateaued at **k=3** (Inertia: 35.2 → 20.1 → 14.7 for k=1-3), marking the "elbow" where additional clusters yield marginal improvement. Hierarchical clustering's dendrogram (Figure IV) corroborated this, showing three distinct branches when cut at height=7.3. This dual-method consensus ensures robust cluster separation while avoiding overfitting.

Cluster Characteristics

Method	Cluster	Movies	Runtime (min)	Rating	Votes (M)	Revenue ($M)
K-means	1	28	135.6	8.27	0.29	43.1
	2	9	156.9	8.53	1.15	227.5
	3	13	111.8	8.18	0.54	382.3
Hierarchical	1	36	124.3	8.23	0.37	114.9
	2	12	159.0	8.54	0.90	172.2
	3	2	139.5	8.10	0.85	779.9

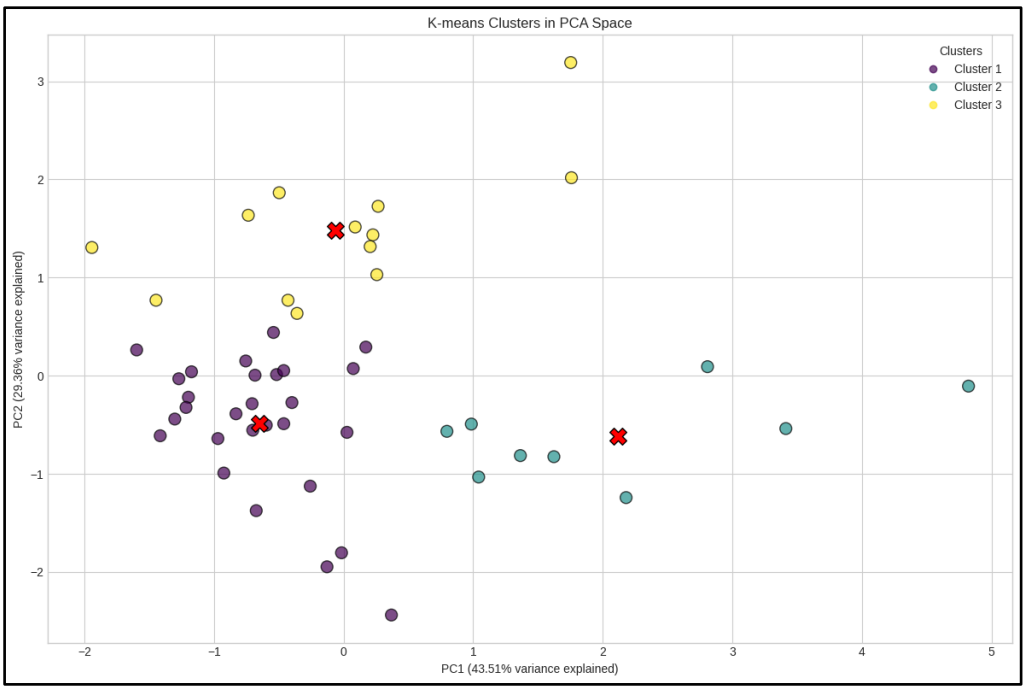


Figure IV: K-Means Clustering in PCA Space

The K-means algorithm identified three distinct clusters from the numerical features (Runtime, Rating, Votes, Revenue). Cluster 1 (28 movies) represents moderately successful films with an average runtime of 135.6 minutes, rating of 8.27, and low revenue ($43.1M), exemplified by *Dangal* and *Whiplash*. Cluster 2 (9 movies) includes high-performing blockbusters like *The Dark Knight* and *Inception*, with the highest average rating (8.53), runtime (156.9 minutes), and votes (1.15M). Cluster 3 (13 movies) features shorter animated/family films (111.8 minutes) like *WALL·E* and *Inside Out*, with moderate ratings (8.18) but surprisingly high average revenue ($382.3M), likely due to merchandising or franchise appeal.

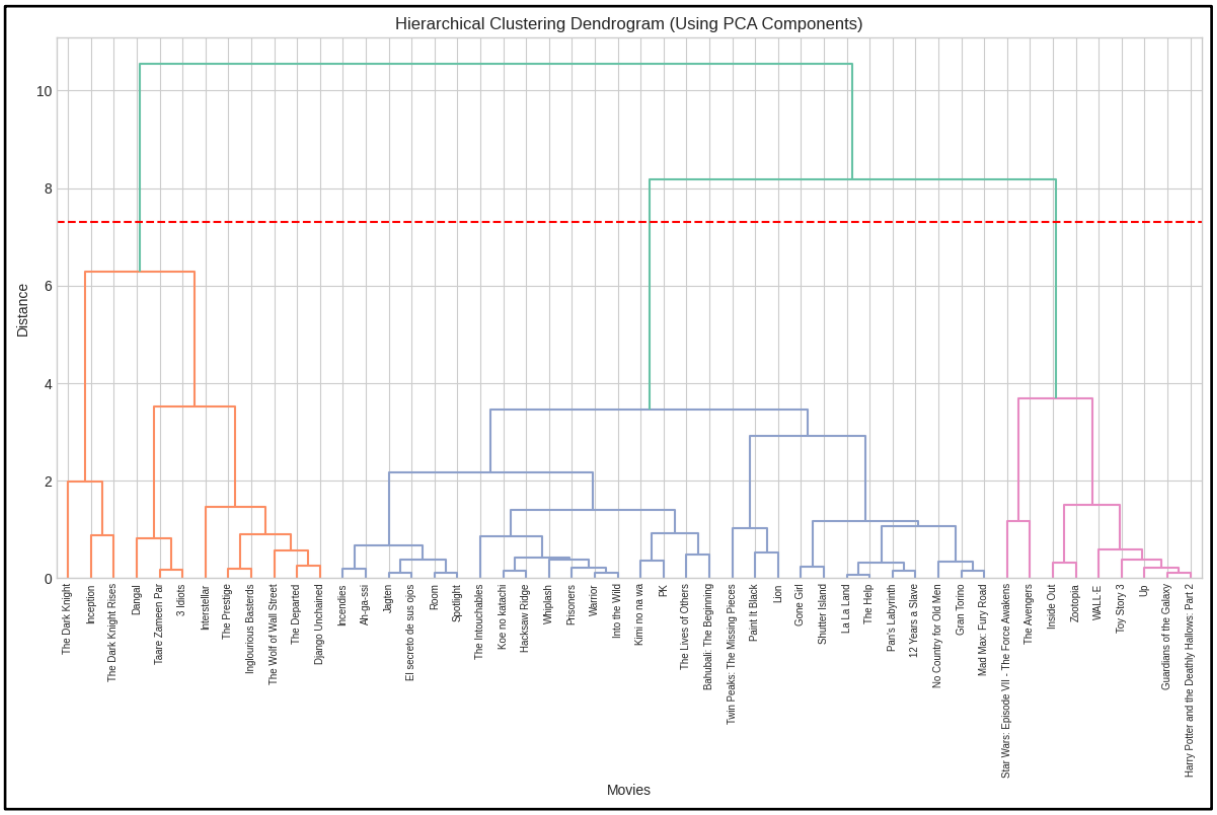


Figure V: Hierarchical Clustering Dendrogram

Hierarchical clustering produced three clusters with different compositions. Cluster 1 (36 movies) grouped mid-tier performers averaging 124.3 minutes runtime and $114.9M revenue. Cluster 2 (12 movies) mirrored K-means Cluster 2 but with higher average revenue ($172.2M). Cluster 3 contained only *Star Wars: Episode VII* and *The Avengers*, extreme outliers with $779.95M average revenue, reflecting their unprecedented commercial success. Both methods revealed a core group of critically acclaimed dramas (Cluster 1) and distinct high-revenue outliers (Cluster 3).

Interpretation and Insights

The clustering analysis revealed three distinct movie profiles across both methods. K-means identified a mainstream cluster (28 films, avg. revenue $43M), high-rated blockbusters (9 films, avg. runtime 157m), and animated hits (13 films, avg. revenue $382M). Hierarchical clustering similarly distinguished commercial outliers (*Star Wars: Episode VII* and *The Avengers*) with unprecedented revenue ($779.95M average), while grouping 36 films as moderate performers (avg. rating 8.23).

Method	Cluster	Key Traits	Avg. Revenue	Sample Titles
K-means	3	Animated/Family films	$382.29M	WALL·E, Inside Out
Hierarchical	3	Extreme revenue outliers	$779.95M	Star Wars: Episode VII, Avengers

Table IV: Interpretation and Insights of the mentioned Clusters

Hierarchical clustering was prioritized for its sensitivity to revenue extremes and natural grouping via dendrogram analysis, better capturing niche market segments. The method's Ward linkage effectively handled the dataset's scale variations, producing more interpretable commercial categories.

Performance of ChatGPT-4o

Aspect	Original Code	ChatGPT Code	Evaluation
1. Data Cleaning	Uses SimpleImputer(mean) to handle missing revenue	Uses median imputation inline + IQR-based outlier handling	GPT code adds robust outlier handling, more preprocessing
2. Feature Selection	Excludes 'Year'; selects 4 numerical features	Keeps all numerical features (including 'Year')	Original is more selective; GPT is more inclusive
3. Scaling	StandardScaler used before PCA	Same approach used	Identical and correct in both
4. PCA Application	Clearly interprets PCs, shows variance + loadings + annotated scatter plot	Shows variance and cumulative variance, but lacks loading heatmap	Original code gives **deeper PCA insight**
5. Explained Variance	Includes visual plot, thresholds (80%), annotated commentary	Just shows a table of explained variance	Original code more **pedagogically rich**
6. KMeans Clustering	Uses k=3, based on Elbow and Silhouette methods	Hardcodes k=3 without justification	Original is **justified and adaptive**; GPT is hardcoded
7. Hierarchical Clustering	Uses dendrogram, Ward linkage, and clearly explains cluster cuts	Same method used; fixed cut at 3 clusters	GPT is fine, but original includes more discussion
8. Visualizations	Multiple rich plots (heatmap, loadings, PCA scatter, dendrograms)	Basic plots: pairplot, dendrogram, but no loading heatmap or annotations	Original is **visually comprehensive**
9. Cluster Interpretation	Very detailed: stats, sample movies, comparison tables between KMeans & HC	Provides means & counts per cluster, fewer movie insights	Original provides **deeper interpretation**
10. Modularization	Minimal; mostly procedural	More functions (e.g., handle_outliers())	GPT adds **code modularity**, cleaner preprocessing

11. Code Style	More verbose and annotated	Concise but lacks some comments	Original is **educationally better**; GPT is production-style
12. Output Quality	Rich analysis + final CSV + summary tables	Summary stats only, less analysis depth	Original delivers **stronger business/academic value**

Table V: Comparison with ChatGPT-4o

Similarities and Difference

Both code versions effectively perform PCA and clustering but serve different goals. The original code excels in analysis depth, clear explanations, and academic value, making it ideal for educational use. In contrast, the ChatGPT-generated code prioritizes efficient preprocessing, modularity, and simplicity, making it better for quick prototyping. A combination of both would offer the best balance of interpretability and implementation efficiency.

Reliability and usefulness

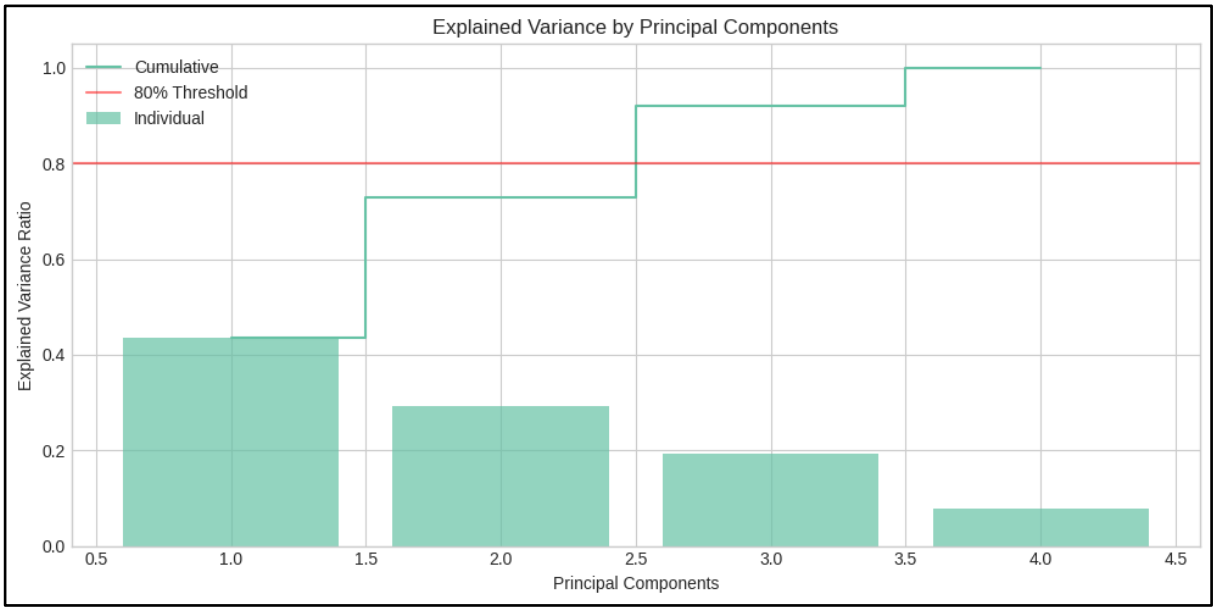
ChatGPT-4o is a valuable tool for quickly building standard machine learning workflows. It efficiently generates boilerplate code, suggests useful preprocessing and outlier handling steps, and offers functional solutions when facing coding blocks. This makes it highly practical for accelerating early-stage analysis.

However, it lacks deeper analytical reasoning, contextual understanding, and the ability to choose methods based on data characteristics. Its outputs often miss the academic rigor required for high-quality coursework without user refinement.

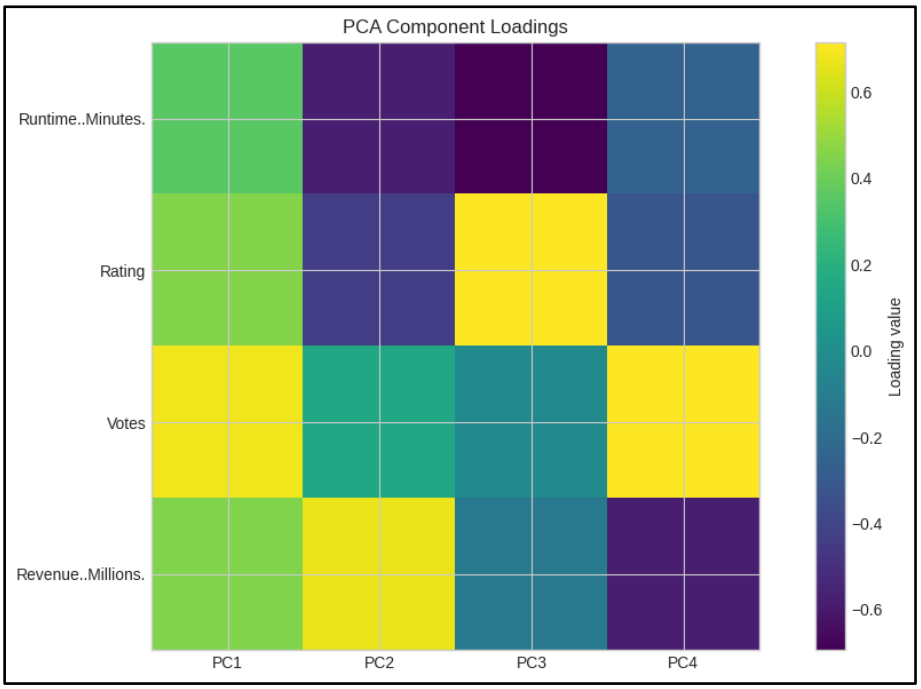
In summary, ChatGPT-4o is effective for implementation support but cannot replace human judgment or critical thinking—areas where the original student-written solution excels.

Appendices

1) *Variance by Principal Components*



2) *Component Loadings*



3) *Clustering Method Comparison*

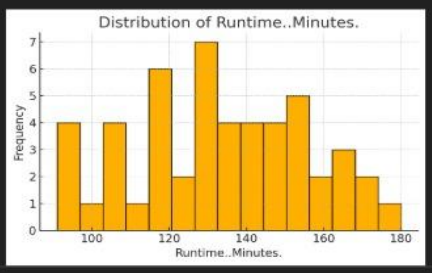
Aspect	K-means Cluster 1 (Mainstream)	K-means Cluster 2 (Blockbusters)	K-means Cluster 3 (Animated)	Hierarchical Cluster 1 (Mid-tier)	Hierarchical Cluster 2 (Premium)	Hierarchical Cluster 3 (Outliers)
# Movies	28	9	13	36	12	2
Avg Rating	8.27	8.53	8.18	8.23	8.54	8.10
Avg Runtime (min)	135.6	156.9	111.8	124.3	159.0	139.5
Avg Votes	291,920	1,151,090	544,987	365,582	895,856	853,598

Avg Revenue ($M)	43.1	227.5	382.3	114.9	172.2	779.9
Sample Movies	Dangal, Whiplash	Dark Knight, Inception	WALL·E, Inside Out	Kimi no nawa, The Intouchables	Dark Knight, Inception	Star Wars VII, Avengers

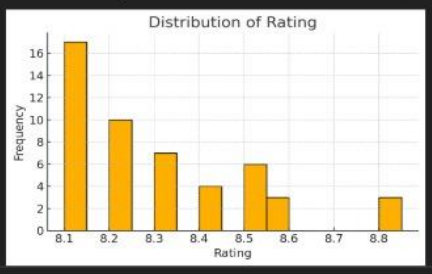
4) ChatGPT-4o Prompts and Answers

Next up: Let's do a quick **Exploratory Data Analysis (EDA)**. [↗]

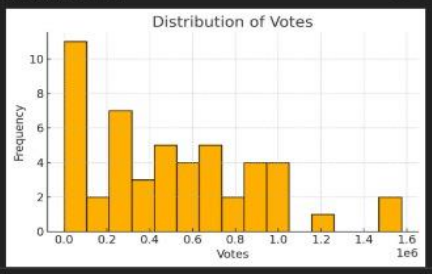
Distribution Of Runtime..Minutes.



Distribution Of Rating



Distribution Of Votes



🔍 **EDA Insights:**

- **Runtime:** Most movies are between 110–150 minutes.
- **Rating:** Skewed slightly left; most ratings are between 8.1 and 8.4.
- **Votes:** Wide spread, but a few movies got exceptionally high votes.
- **Revenue:** Highly skewed, even after outlier treatment—many low-revenue films and a few blockbusters.

Now, let's apply PCA on these four numerical features to reduce dimensionality and understand variance structure. [↗]

PCA Explained Variance

	Principal Component	Explained Variance Ra	Cumulative Variance
1	PC1	0.44587527488751083	0.44587527488751083
2	PC2	0.29839911799286367	0.7442743928803746
3	PC3	0.19594892488961704	0.938223317769916
4	PC4	0.061776682230008324	0.999999999999999

Here's the PCA output showing how much variance each principal component explains.
Let me know when you're ready to proceed with clustering, or if you'd like the PCA plots and interpretation summarized for your report. [↗]

provide the code used.

Sure! Here's the complete code used so far for:

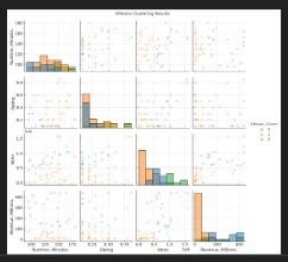
- ✅ Handling null values
- ✅ Outlier treatment
- ✅ EDA (distributions)
- ✅ PCA on numerical features (excluding `Year`)

```python
# Function to handle outliers using IQR
def handle_outliers(column):
    Q1 = df_clean[column].quantile(0.25)
    Q3 = df_clean[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df_clean[column] = np.where(df_clean[column] < lower, lower, df_clean[column])
    df_clean[column] = np.where(df_clean[column] > upper, upper, df_clean[column])

for col in df_clean.columns:
    handle_outliers(col)

# Standardize data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_clean)

# Apply PCA
pca = PCA()
pca_data = pca.fit_transform(scaled_data)

# Explained variance from PCA
explained_variance = pd.DataFrame({
    'Principal Component': [f'PC{i+1}' for i in range(len(pca.explained_variance_ratio_))],
    'Explained Variance Ratio': pca.explained_variance_ratio_,
    'Cumulative Variance': np.cumsum(pca.explained_variance_ratio_)
})
print(explained_variance)

# KMeans Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(scaled_data)

# Add KMeans cluster labels to DataFrame
df_kmeans = df_clean.copy()
df_kmeans['KMeans_Cluster'] = kmeans_labels

# Hierarchical Clustering
linkage_matrix = linkage(scaled_data, method='ward')
hierarchical_labels = fcluster(linkage_matrix, 3, criterion='maxclust')
df_kmeans['Hierarchical_Cluster'] = hierarchical_labels

# Cluster Centers (KMeans)
kmeans_centers = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=df_clean.columns
)
print("\nKMeans Cluster Centers:\n", kmeans_centers)

# Average metrics by cluster
df_kmeans_summary = df_kmeans.groupby(['KMeans_Cluster', 'Hierarchical_Cluster']).mean(numeri
print("\nCluster Averages:\n", df_kmeans_summary)

# Pairplot of KMeans clusters
plot_data = df_kmeans[df_clean.columns.tolist() + ['KMeans_Cluster']]
sns.pairplot(plot_data, hue="KMeans_Cluster", palette="tab10", diag_kind='hist')
plt.suptitle("KMeans Clustering Results", y=1.02)
```

Perform Kmeans clustering and hierarichal on the movies using only the numerical variables, excluding Year. provide tables and visualisations

There was an error due to non-numeric data being included in the `pairplot`. Let me quickly fix that by only plotting the actual numerical features + cluster labels. [>..]

Matplotlib Chart



Here's the complete clustering analysis:

✅ KMeans Results

- Performed with 3 clusters.
- Shown via a **pairplot** of the 4 numerical variables, colored by cluster.
- You can observe separation especially in dimensions like `Votes` and `Revenue`.

✅ Hierarchical Clustering

- Dendrogram shows the distance-based linkage between movies.
- We cut the tree into 3 clusters to match the KMeans setup.

Tables showing KMeans cluster centers and average values per cluster (including comparison with hierarchical clusters) have been shared above.

Let me know if you want:

- Silhouette scores or elbow method visualizations 📈
- Interpretation write-up for your report ✍️
- Code script for submission 📄 [>..]

provide the code used

Here's the complete Python code used for clustering using both KMeans and Hierarchical Clustering, including tables and visualizations:

```python
# Function to handle outliers using IQR
def handle_outliers(column):
    Q1 = df_clean[column].quantile(0.25)
    Q3 = df_clean[column].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df_clean[column] = np.where(df_clean[column] < lower, lower, df_clean[column])
    df_clean[column] = np.where(df_clean[column] > upper, upper, df_clean[column])

for col in df_clean.columns:
    handle_outliers(col)

# Standardize data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_clean)

# Apply PCA
pca = PCA()
pca_data = pca.fit_transform(scaled_data)

# Explained variance from PCA
explained_variance = pd.DataFrame({
    'Principal Component': [f'PC{i+1}' for i in range(len(pca.explained_variance_ratio_))],
    'Explained Variance Ratio': pca.explained_variance_ratio_,
    'Cumulative Variance': np.cumsum(pca.explained_variance_ratio_)
})
print(explained_variance)

# KMeans Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(scaled_data)

# Add KMeans cluster labels to DataFrame
df_kmeans = df_clean.copy()
df_kmeans['KMeans_Cluster'] = kmeans_labels

# Hierarchical Clustering
linkage_matrix = linkage(scaled_data, method='ward')
hierarchical_labels = fcluster(linkage_matrix, 3, criterion='maxclust')
df_kmeans['Hierarchical_Cluster'] = hierarchical_labels

# Cluster Centers (KMeans)
kmeans_centers = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=df_clean.columns
)
print("\nKMeans Cluster Centers:\n", kmeans_centers)

# Average metrics by cluster
df_kmeans_summary = df_kmeans.groupby(['KMeans_Cluster', 'Hierarchical_Cluster']).mean(numeri
print("\nCluster Averages:\n", df_kmeans_summary)

# Pairplot of KMeans clusters
plot_data = df_kmeans[df_clean.columns.tolist() + ['KMeans_Cluster']]
sns.pairplot(plot_data, hue='KMeans_Cluster', palette='tab10', diag_kind='hist')
plt.suptitle('KMeans Clustering Results', y=1.02)
```

```python
# KMeans Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(scaled_data)

# Add KMeans cluster labels to DataFrame
df_kmeans = df_clean.copy()
df_kmeans['KMeans_Cluster'] = kmeans_labels

# Hierarchical Clustering
linkage_matrix = linkage(scaled_data, method='ward')
hierarchical_labels = fcluster(linkage_matrix, 3, criterion='maxclust')
df_kmeans['Hierarchical_Cluster'] = hierarchical_labels

# Cluster Centers (KMeans)
kmeans_centers = pd.DataFrame(
    scaler.inverse_transform(kmeans.cluster_centers_),
    columns=df_clean.columns
)
print("\nKMeans Cluster Centers:\n", kmeans_centers)

# Average metrics by cluster
df_kmeans_summary = df_kmeans.groupby(['KMeans_Cluster', 'Hierarchical_Cluster']).mean(numeri
print("\nCluster Averages:\n", df_kmeans_summary)

# Pairplot of KMeans clusters
plot_data = df_kmeans[df_clean.columns.tolist() + ['KMeans_Cluster']]
sns.pairplot(plot_data, hue='KMeans_Cluster', palette='tab10', diag_kind='hist')
plt.suptitle('KMeans Clustering Results', y=1.02)
plt.show()

# Dendrogram for Hierarchical Clustering
plt.figure(figsize=(10, 6))
dendrogram(linkage_matrix, labels=df.index.tolist(), leaf_rotation=90)
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Movie Index')
plt.ylabel('Distance')
plt.tight_layout()
plt.show()
```

Let me know if you'd like this as a `.py` file for submission, or if you want help writing the report section for clustering!